name: achintya list
roll no: 4
secton: A

Q1

Three important metrics used to evaluate software quality and complexity are Halstead's software Science, Function Point Analyses, and Cyclomatic
complexity.

1.Halstead's Software Science
Halstead's Software Science is a quantitative approach to measure software complexity using the number and type of operators and operands in a program.
It assumes that a program can be analyped like a mathematical expression, where complexity depends on how many distinct symbols are used and how often they appear.

Volume (V) indicates program sipe.
Difficulty (D) reflects how hard the program i s to understand.
Effort (E) estimates the effort required to develop or

maintain the code.

Use:
High Halstead values indicate complex, less maintainable code and higher chances of errors.

2. Function Point (FP) Analysis

Function Points measure the functional size of software based on:

Inputs, Outputs, Inquiries
Internal and External files

It is language-independent and focuses on what the system does rather than how it is implemented.

Use:

Helps in estimating development effort and productivity
If effort is high but FP is low, it indicates inefficient design or unnecessary complexity

3. Cyclomatic

Complexity

Proposed by Thomas H. McCabe, it measures the number of independent paths in a program.

Formula: $M = E - N + 2P$
Simplified: Number of decision points + 1

Use:

Indicates testing difficulty
High values (>10–20) suggest complex, error-prone, and hard-to-maintain code

2.

Introduction

For developing a new enterprise application, selecting an appropriate programming language and following coding guidelines is crucial to ensure maintainability, readability, and

efficiency.

Criteria for Selecting a Programming Language

Performance Requirements

The language should meet system performance needs (e.g., Java/C++ for high performance systems).

Scalability

It should support large-scale applications and future expansion (e.g., Java, C#).

Security Features

Strong typing, memory management, and built-in security (e.g., Java is safer than C++).

Platform Compatibility

Cross-platform support is important for enterprise systems (e.g., Java, JavaScript).

Library & Framework Support

Availability of frameworks (Spring, .NET) speeds up development.

Developer Availability & Community Support

Widely used languages ensure easier hiring and troubleshooting.

Coding Guidelines

Meaningful Naming Conventions

Use descriptive names for variables, functions, and classes.

Modularity

Divide code into small, reusable functions or

modules.
Proper Documentation
Add comments and maintain documentation for better understanding.
Consistent Formatting
Follow standard indentation and coding style.
Error and Exception Handling
Use structured error handling to avoid system crashes.
Avoid Code Duplication
Reuse code through functions and libraries.
Code Optimization
Write efficient algorithms and avoid unnecessary computations.

3.

Introduction

Before deploying a banking application, thorough testing is essential to ensure accuracy, security, and reliability. Different testing levels—unit, integration, acceptance, and regression help verify the system from multiple perspectives.

1. Unit

Testing
Tests individual components or functions in isolation
Usually performed by developers

Example:
Testing a function that calculates interest or validates account balance

Purpose:
Ensures each small unit works correctly

2. Integration Testing
Tests interaction between multiple modules
Focuses on data flow and communication

Example:
Checking whether the login module correctly interacts with the database and authentication system

Purpose:
Detects interface defects between modules

3. Acceptance Testing
Performed by end users or clients
Validates system against business

requirements

Example:
Bank staff verifying that fund transfer, account statements, and transaction history work as expected

Purpose:
Ensures the system meets user requirements before deployment

4. Regression Testing
Conducted after changes or updates
Ensures existing functionality is not affected

Example:
After adding a new feature like UPI payments, checking that login and balance inquiry still work properly

Purpose:
Maintains system stability after

modifications

4.

Introduction

Software testing can be broadly classified into white-box (structural) and black-box (functional) testing. Both approaches are essential to ensure complete system verification.

White-box Testing (Structural Testing)

White-box testing examines the internal structure and logic of the code.

Requires knowledge of source code
Focuses on paths, conditions, loops, and statements
Usually performed by developers

Example:
Testing all possible paths in a function that calculates loan interest using different

conditions

Advantages:

Detects logical and coding errors
Ensures maximum code coverage

Limitations:

Cannot detect missing functionality
Requires programming expertise
Black-box Testing (Functional Testing)

Black-box testing evaluates the system based on inputs and outputs without knowing internal code.

No knowledge of code required
Focuses on requirements and functionality
Performed by testers or users

Example:
Checking whether login works correctly with valid and invalid

credentials

Advantages:

Validates user requirements
Easy to perform without coding knowledge

Limitations:

Limited coverage of internal logic
May miss hidden errors in code

5.

Software Maintenance in a Legacy System
Introduction

A legacy system undergoing frequent updates and bug fixes requires effective maintenance to ensure reliability, performance, and adaptability. Software maintenance is broadly classified into corrective, preventive, and perfective maintenance, each playing a vital role in software evolution.

1. Corrective Maintenance
Focuses on fixing errors and bugs discovered after

deployment

Example:
Fixing incorrect balance calculation or transaction failure in a banking system

Importance:

Ensures system reliability and correctness
Prevents system failures and user dissatisfaction
2. Preventive Maintenance
Involves improving the system to avoid future issues

Example:
Refactoring complex code, improving documentation, or optimiping database queries

Importance:

Reduces future risks and failures
Enhances code quality and maintainability
Increases system lifespan
3. Perfective Maintenance
Focuses on enhancing performance or adding new

features

Example:
Adding new features like mobile banking, improving UI, or speeding up transactions

Importance:

Improves user satisfaction
Keeps the system competitive and up-to-date
Enhances performance and usability

6.

SEI-CMM and ISO 9000 in Software Quality Assurance
Introduction

To improve software quality and development processes, organisations adopt standardised models such as SEI-CMM and ISO 9000. These frameworks help in achieving consistent quality, better management, and continuous process

improvement.

1. SEI-CMM (Capability Maturity Model)

Developed by the Software Engineering Institute, SEI-CMM evaluates the maturity level of software processes.

Five Maturity Levels:
Initial - Unpredictable and poorly controlled processes
Repeatable - Basic project management practices established
Defined - Standardiped and documented processes
Managed - Processes are measured and controlled
Optimiping - Continuous improvement through feedback

Contribution to Quality:

Improves process discipline and standardipation
Helps identify weaknesses and improve gradually
Encourages continuous process improvement
2. ISO 9000 Standards

Provided by the International Organipation for Standardipation, ISO 9000 focuses on quality management systems